

Reviewer Pack — Minimal Rank of Tropicalized Quaternion Algebras vs Monotone...

Entry 9e8eb5d29063 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 03:42:56 UTC

Minimal Rank of Tropicalized Quaternion Algebras vs Monotone k-CLIQUE Circuit Size

Entry ID: 9e8eb5d29063 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 03:42:56 UTC

1. Conjecture

Field A (mathematical branch): Quaternionic Algebra Theory **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity (for k-CLIQUE)

Statement:

[‘For every k-CNF instance with n variables, the minimal rank of the tropicalized quaternion algebra corresponding to its negation Cayley representation is $\Theta(n^{\{1/2-k\}})$ ’, ‘where the minimal rank is defined as the minimum dimension over all possible quaternions that can represent the given CNF.’, ‘Additionally, for any k-CNF instance with n variables, there exists a monotone k-CLIQUE circuit of size at most $2^{\Theta(n^{\{1/2-k\}})}$ ’]

Rationale (proposer’s reasoning):

[‘Quaternionic algebra theory provides a unique representation that can potentially capture the essence of Boolean functions in a more geometric and algebraic way.’, ‘The conjecture proposes a link between the algebraic structure of quaternions and the circuit complexity, which has not been explored before in the context of k-CLIQUE.’, ‘This connection could reveal new insights into the nature of boolean functions and potentially lead to improved algorithms for monotone k-CLIQUE.’]

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c073ce5ea0b6ad68

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for at least 80% of the 30 random seeds, the computed minimal rank of the tropicalized quaternion algebra for a k-CNF instance with n variables is within $\pm 10\%$ of $\Theta(n^{\{1/2-k\}})$ and the size of the smallest monotone k-CLIQUE circuit is at most $2^{\Theta(n^{\{1/2-k\}})}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def negation_cayley_representation(cnf):
    return sum(abs(sum clause)) for clause in cnf

def tropicalized_quaternion_rank(negated_cayley):
    # Placeholder for actual computation, replace with actual algorithm
    return len(negated_cayley)

def monotone_k_clique_size(rank):
    # Placeholder for actual computation, replace with actual algorithm
    return 2 ** rank

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    cnf = [[random.choice([-1, 1]) * (i + 1) for i in range(n)] for _ in range(n)]

    negated_cayley = negation_cayley_representation(cnf)
    rank = tropicalized_quaternion_rank(negated_cayley)
    clique_size = monotone_k_clique_size(rank)

    expected_rank = n ** (1/2 - 1) # Simplified for demonstration
    expected_clique_size = 2 ** expected_rank

    metric_value = rank
    conjecture_holds = abs(rank - expected_rank) <= 0.1 * expected_rank and clique_size <= expected_clique_size
    counterexample = "" if conjecture_holds else f"rank={rank}, expected_rank={expected_rank}, clique_size={clique_size}, expected_clique_size={expected_clique_size}"

    return {
        "metric_name": "Minimal Rank",
```

```

        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 97) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    elif any(r["counterexample"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result["counterexample"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE not_enough_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_920ca07b.py", line 60, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_920ca07b.py", line 36, in run_trial
    rank = tropicalized_quaternion_rank(negated_cayley)
           ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_920ca07b.py", line 23, in tropicalized_quaternion_rank
    return len(negated_cayley)
           ~~~~~
TypeError: object of type 'int' has no len()

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Investigate and fix the error in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12838
2	propose	ollama_remote	glm4:latest	0	0	12417
3	preregistration	ollama_remote	glm4:latest	0	0	6216
4	novelty	ollama_remote	glm4:latest	0	0	5095
5	novelty	ollama_remote	glm4:latest	0	0	5164
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15993
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8720
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9713
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7649
10	judge	ollama_remote	glm4:latest	0	0	8089

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 91894 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/9e8eb5d29063.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/9e8eb5d29063.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/9e8eb5d29063.tar.gz (if generated)